A*-Based Autonomous Drone Pathfinding and Dynamic Obstacle Avoidance in 2D Grid Environments

Onyekachi Onyenokwe
Department of Computer Science
Full Sail University
Florida, USA
oonyenokwe@student.fullsail.edu

***Abstract*—This paper presents the design, implementation, and evaluation of an autonomous drone navigation system operating in a two-dimensional grid environment. The system applies the A* search algorithm with a Manhattan-distance heuristic to calculate efficient routes around randomly generated static obstacles. A virtual sensor map monitors moving obstacles and enables the autonomous controller to pause and recalculate its route when a collision threat is detected. The implementation includes an animated visualization, trajectory tracking, controlled hazard injection, and automated performance measurement. Evaluation across 10 successful trials with different reproducible grid configurations produced an average path efficiency of 96.14%, with a standard deviation of 4.39%. The drone completed each accepted trial in an average of 39.70 simulation steps and performed one successful reroute per trial. One additional configuration was excluded because no valid initial route existed. These results demonstrate that A* can support efficient and reproducible path planning and dynamic obstacle avoidance in a simulated two-dimensional environment.**

***Keywords*—A* search, autonomous navigation, dynamic obstacle avoidance, grid-based simulation, heuristic search, path planning, unmanned aerial vehicles.**

I. Introduction

Autonomous unmanned aerial vehicles (UAVs) require reliable path-planning and obstacle-avoidance capabilities to navigate complex environments safely. A navigation system must identify an efficient route between its starting position and destination while accounting for static barriers and responding to hazards that enter the planned trajectory. Recent research continues to evaluate A*-based methods because they offer systematic and computationally efficient path search within grid-represented environments [1], [2]. Dynamic environments introduce an additional challenge because a previously valid route may become unsafe as obstacles move, requiring the vehicle to detect the threat and recalculate its path [3].

This project implements a Python-based autonomous drone simulation in a 20 × 20 two-dimensional grid. The system uses A* search with a Manhattan-distance heuristic, virtual obstacle sensing, controlled dynamic-hazard injection, and autonomous pause-and-reroute logic. Its performance is evaluated through trajectory length, path efficiency, reroute frequency, hazard pauses, and algorithm execution time. A multi-trial benchmark across reproducible randomly generated environments is also conducted to assess the consistency of the navigation approach.

II. System Architecture and Algorithm Design

A. Environment and Perception Model

The simulation represents the operating environment as a 20 × 20 discrete grid containing traversable cells, static obstacles, a starting position, and a destination. Static obstacles occupy approximately 20% of the grid and are generated using reproducible random seeds. Drone movement is restricted to four cardinal directions: up, down, left, and right. Two dynamic obstacles move through the environment and reverse direction when they reach a grid boundary.

At each update cycle, the system constructs a temporary sensor map containing the current static and dynamic obstacle positions. The perception logic then evaluates the drone's next planned cell. If a dynamic obstacle occupies that cell, the controller classifies the movement as a collision threat and prevents the drone from entering the unsafe position. A controlled hazard is introduced at simulation step 8 to provide a reproducible test of the detection and rerouting process.

B. A* Pathfinding Engine

The navigation system uses A* search to calculate a collision-free path between the drone's current position and the destination. A* evaluates each candidate node $n$using the cost function [1], [2]:

$$f(n) = g(n) + h(n) \tag{1}$$

where $g(n)$ is the accumulated movement cost from the starting node to n, and $h(n)$estimates the remaining cost from n to the destination. Each cardinal movement has a cost of one grid unit. Because movement is limited to four directions, the system uses the Manhattan-distance heuristic:

$$h(n) = |x_n - x_g| + |y_n - y_g| \tag{2}$$

where (x_n, y_n) represents the candidate node and (x_g, y_g) represents the destination. Candidate nodes are maintained in a priority queue and processed according to their estimated total cost. Obstacle cells are excluded from

expansion, and the search terminates when the destination is reached or no valid path remains.

C. Autonomous Decision Logic and Rerouting

When the virtual sensor map identifies a dynamic obstacle in the drone's next planned cell, the controller performs the following sequence:

1. **Threat detection:** The unsafe movement is rejected and recorded as a hazard pause.
2. **Map update:** The current dynamic-obstacle positions are marked as impassable in the temporary grid.
3. **Path recalculation:** A* is executed again from the drone's current position to the destination.
4. **Route continuation:** If a valid alternative path is found, the reroute count is incremented and the drone advances using the new safe route.
5. **Holding behavior:** If no alternative route is available, the drone remains in its current position until a later update cycle.

This process enables the drone to respond to changes in the simulated environment without entering a known obstacle cell. Recent UAV research similarly identifies repeated sensing, map updating, and path recalculation as central requirements for navigation in dynamic environments [3]–[5].

Listing 1 Dynamic obstacle detection and autonomous A* rerouting logic.

```
next_step = self.path[1]

if next_step in dynamic_positions:
    self.pause_count += 1

    new_path = astar(
        temporary_grid,
        self.drone_position,
        self.goal,
    )

    if new_path and len(new_path) > 1:
        self.path = new_path
        self.reroute_count += 1
        next_step = self.path[1]
    else:
        return

self.drone_position = next_step
```

The complete simulation source code, benchmark implementation, generated results, and supporting evidence are available in the project repository [6].

III. Experimental Setup and Results

A. Evaluation Procedure

The system was evaluated through a reproducible single-run demonstration and a multi-trial benchmark. The demonstration used random seed 42 to maintain a consistent static-obstacle configuration and controlled dynamic-hazard event. During execution, the system recorded total simulation steps, reroute count, hazard pauses, optimal grid distance, actual trajectory length, path efficiency, and algorithm execution time.

For broader evaluation, the benchmark attempted simulations using sequential random seeds until 10 valid trials were completed. Each trial used a different reproducible static-obstacle arrangement while preserving the 20 × 20 grid, 20% obstacle-generation probability, start and destination positions, and controlled hazard event. A maximum of 500 update cycles was permitted per trial. One additional seed was rejected because its generated environment had no valid initial route.

B. Navigation and Rerouting Behavior

During the visual demonstration, the drone initially followed the route calculated by A*. At step 8, a moving obstacle entered its next planned cell. The controller rejected the unsafe movement, updated the temporary obstacle map, and recalculated the route from the drone's current position. The drone subsequently followed the revised path and reached the destination without entering an occupied cell. Figure 1 shows the completed trajectory, including the original route representation, static obstacles, dynamic hazards, and destination.

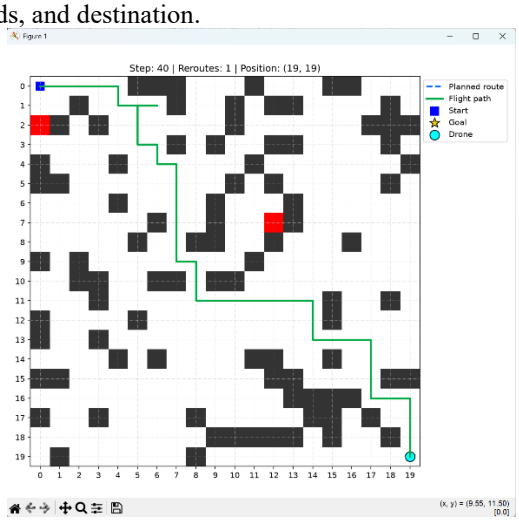

Fig. 1. *Completed autonomous drone trajectory following dynamic obstacle detection and A** rerouting.

The trajectory heatmap in Fig. 2 provides a spatial representation of the grid cells visited during the demonstration. The result confirms that the drone maintained a continuous route from the starting position to the destination while avoiding blocked cells.

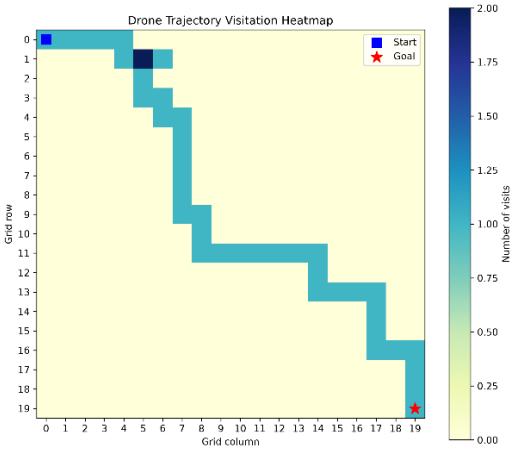

Fig. 2. *Grid-cell visitation heatmap for the completed drone trajectory.*

C. Multi-Trial Benchmark Results

Table I summarizes the results from the 10 successful benchmark trials.

TABLE I. MULTI-TRIAL AUTONOMOUS NAVIGATION RESULTS

Metric	Measured result
Successful trials	10
Rejected grid configurations	1
Average total steps	39.70
Average reroutes	1.00
Average hazard pauses	1.10
Average path efficiency	96.14%
Path-efficiency standard deviation	4.39%
Average algorithm execution time	0.000790 s

The benchmark attempted 11 grid configurations. Ten configurations contained a valid initial route, and all 10 completed trials reached the destination. One configuration was excluded because no valid initial route existed. Across the 10 completed trials, the average path efficiency was 96.14%, with a standard deviation of 4.39%, indicating that the calculated trajectories generally remained close to the minimum Manhattan distance despite static obstacles and dynamic rerouting. Each successful trial performed one completed reroute in response to the controlled hazard. The average hazard-pause count was slightly higher at 1.10 because one trial required an additional update cycle before a safe alternative path became available.

The mean algorithm execution time was 0.000790 s. This measurement represents the computational update loop without the 300-ms visualization interval and should therefore not be interpreted as the total duration of the animated demonstration. Figure 3 compares the total steps, reroute count, and path efficiency across the individual benchmark trials.

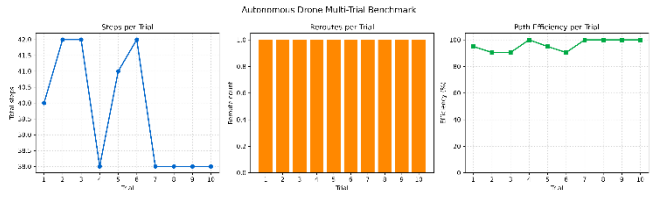

Fig. 3. *Total steps, reroute frequency, and path efficiency across 10 successful benchmark trials.*

IV. DISCUSSION AND FUTURE WORK

The evaluation demonstrates that A* search combined with stepwise virtual obstacle detection can support consistent navigation and rerouting in a simplified two-dimensional grid. Across the 10 completed benchmark trials, the controller reached the destination while maintaining an average path efficiency of 96.14%. These results indicate that full-route recalculation is computationally practical for the evaluated 20 × 20 environments. However, the results should be interpreted within the constraints of the simulation rather than as evidence of real-world UAV reliability.

The system assumes known grid occupancy, unit movement costs, deterministic obstacle motion, and a point-sized drone capable of moving in four cardinal directions. It does not model altitude, acceleration, turning radius, energy consumption, communication delays, sensor uncertainty, or physical collision boundaries. Additionally, the controlled step-8 hazard confirms that the rerouting logic functions correctly, but it does not represent the full unpredictability of moving obstacles in real operating environments. Grid-based A* paths may also contain abrupt directional changes that would be unsuitable for direct execution by a physical UAV [1], [3].

Future work should extend the environment to three dimensions and incorporate UAV kinematic constraints, obstacle-velocity prediction, noisy sensor measurements, and trajectory smoothing. Comparative experiments could evaluate A* against Dijkstra's algorithm, D* Lite, RRT*, and artificial-potential-field approaches using identical obstacle configurations [4], [5]. More advanced extensions could investigate the Dynamic Window Approach or reinforcement learning for continuous control. Increasing the number of trials and testing multiple obstacle densities, grid dimensions, hazard timings, and start–goal configurations would provide a stronger assessment of scalability and robustness.

V. CONCLUSION

This project demonstrated an A*-based autonomous drone navigation system capable of path planning, dynamic-obstacle detection, and route recalculation within a two-dimensional grid. Across 10 completed benchmark trials, the controller reached the destination with an average path efficiency of 96.14% and a standard deviation of 4.39%. Each trial completed a controlled reroute, confirming that the system could detect an obstruction in its planned path and calculate a safe alternative. Although the simulation does not model the full physical and sensory complexity of real UAV navigation, it provides a reproducible foundation for evaluating autonomous path-planning and obstacle-avoidance logic.

VI. AI TOOL ASSISTANCE AND USAGE DISCLOSURE STATEMENT

Generative AI tools were used to support the development of this project. Google Gemini assisted with the initial simulation concept and code draft, while OpenAI ChatGPT assisted with code refinement, debugging, benchmark design, documentation, and academic writing review. All AI-generated material was manually examined and modified as necessary. The final simulation was executed and tested by the author, who verified the pathfinding, obstacle-detection, rerouting, visualization, and performance-measurement functions. The author remains responsible for the accuracy of the implementation, reported results, and conclusions presented in this paper.

REFERENCES

[1] D. Debnath, F. Vanegas, J. Sandino, A. F. Hawary, and F. Gonzalez, "A review of UAV path-planning algorithms and obstacle avoidance

methods for remote sensing applications," Remote Sens., vol. 16, no. 21, Art. no. 4019, Oct. 2024, doi: 10.3390/rs16214019.

[2] W. Zhang, J. Li, W. Yu, P. Ding, J. Wang, and X. Zhang, "Algorithm for UAV path planning in high obstacle density environments: RFA-star," Front. Plant Sci., vol. 15, Art. no. 1391628, Oct. 2024, doi: 10.3389/fpls.2024.1391628.

[3] L. Xu, M. Xi, R. Gao, Z. Ye, and Z. He, "Dynamic path planning of UAV with least inflection point based on adaptive neighborhood A* algorithm and multi-strategy fusion," Sci. Rep., vol. 15, Art. no. 8563, Mar. 2025, doi: 10.1038/s41598-025-92406-w.

[4] A. H. Ahmad, O. Zahwe, A. Nasser, and B. Clement, "Path planning algorithms for unmanned aerial vehicle: Classification, performance, and implementation," in Proc. 2023 Int. Conf. Electr., Comput., Commun. Mechatronics Eng. (ICECCME), 2023, Art. no. 10252168, doi: 10.1109/ICECCME57830.2023.10252168.

[5] K. S. Almazrouei, A. B. Nassif, and M. AlShabi, "Path-planning and obstacle avoidance algorithms for UAVs: A systematic literature review," in Proc. SPIE 12549, Unmanned Systems Technology XXV, Jun. 2023, Art. no. 125490S, doi: 10.1117/12.2664056.

[6] K. Onyenokwe, "Autonomous drone pathfinding and dynamic obstacle avoidance," GitHub repository, 2026. [Online]. Available: https://github.com/kachionyenokwe/autonomous-drone-pathfinding. [Accessed: Aug. 29, 2026].